

AccessQ

Conception et fonctionnement

Documentation de reprise technique pour futurs developpeurs

Ce document a pour objectif de permettre a une nouvelle equipe de comprendre et maintenir AccessQ sans dependance au developpeur originel. Il explique l'intention fonctionnelle, l'architecture, les fichiers importants, les flux metier, les contrats d'API, les regles de securite, les choix de persistance et les points de vigilance. Il ne remplace pas la lecture du code pour une modification fine, mais il donne la carte mentale necessaire pour entrer rapidement dans le projet.

Le contenu a ete reconstruit a partir du depot actuel: BackendAccessQ, frontendAccessQ et AdministrationAccess. Les noms de routes, modeles, variables et comportements cites correspondent au code observe dans le projet.

Note de reprise

Les accents sont volontairement limites dans certains blocs techniques et dans le corps du PDF pour garantir une generation fiable avec la police PDFKit standard. Les noms de fichiers et champs restent exacts.

Table des matieres

1. Vision produit et parcours utilisateur
2. Cartographie du depot
3. Architecture backend
4. Modele de donnees PostgreSQL/Prisma
5. Authentification, roles et sessions
6. Contrats d'API
7. Flux metier detailles
8. Frontend Next.js
9. Administration super-admin
10. Emails, QR physiques et cartes
11. Configuration, installation et deploiement
12. Tests, logs et diagnostic
13. Maintenance et evolutions recommandees

1. Vision produit et parcours utilisateur

AccessQ est un systeme de controle d'accès par QR code. Une organisation s'inscrit, verifie son email, cree ses zones physiques, cree des evenements associes a ces zones, puis genere des QR codes pour les participants. Le jour de l'evenement, les agents ouvrent l'ecran de scan, lisent les QR codes avec la camera et obtiennent une decision instantanee: acces autorise ou refuse avec une raison.

Le systeme est multi-tenant: chaque organisation possede ses propres utilisateurs, evenements, zones, QR codes et journaux de scans. Le backend doit toujours filtrer par org_id afin qu'un utilisateur d'une organisation ne puisse pas consulter ou modifier les donnees d'une autre organisation.

- Parcours administrateur d'organisation: inscription, verification email, connexion, creation des zones, creation d'evenements, generation/import de QR, invitation d'agents, consultation des statistiques et exports.

- Parcours agent: connexion avec identifiants fournis, acces au scanner, scan d'un QR, lecture immediate de la decision et reprise du scan.
- Parcours super-admin plateforme: connexion a AdministrationAccess, consultation globale, activation/desactivation ou archivage des organisations et utilisateurs.

L'element central n'est pas l'image QR elle-meme mais le jeton unique_token stocke en base. L'image QR encode seulement un petit JSON contenant le token et l'identifiant evenement. Les donnees personnelles du deteneur restent cote serveur.

2. Cartographie du depot

Le depot contient trois applications deployables separement. Elles partagent une base PostgreSQL mais n'ont pas toutes le meme role.

Qr Access 2/	
BackendAccessQ/	API principale Express + Prisma
frontendAccessQ/	Interface utilisateur Next.js
AdministrationAccess/	Panneau super-admin Express + EJS
conception et fonctionnement.pdf	
AccessQ_Investor_Pitch.pptx	Fichier existant non suivi par Git

BackendAccessQ

```
BackendAccessQ/server.js
  Charge dotenv, importe src/app.js, demarre le serveur sur PORT ou 5000.
BackendAccessQ/src/app.js
  Configure trust proxy, helmet, CORS, rate limiter global, parsers, cookies, statics, routes
  et handler d'erreur.
BackendAccessQ/src/routes/
  Mappe les endpoints HTTP vers les controllers.
BackendAccessQ/src/controllers/
  Lit req/res, valide les entrees simples, appelle les services, formate les reponses JSON.
BackendAccessQ/src/services/
  Contient les regles metier et les appels Prisma.
BackendAccessQ/src/middleware/
  Authentification JWT, controle de roles, rate limiting, request logging.
BackendAccessQ/src/prisma/schema.prisma
  Source du schema Prisma principal.
BackendAccessQ/src/statics/
  Fichiers publics servis par Express: qrcodes, cards, logos, templates.
```

frontendAccessQ

Le frontend est une application Next.js App Router. Les pages sont dans frontendAccessQ/src/app. Le fichier lib/api.js est le wrapper d'appel API a privilegier car il gere les cookies, le refresh de session et la redirection login.

AdministrationAccess

AdministrationAccess est une application Express classique avec vues EJS. Elle utilise le meme DATABASE_URL mais une session adminToken signee avec JWT_SECRET. Elle est reservee aux comptes SUPER_ADMIN.

3. Architecture backend

Le backend expose une API REST. Toutes les routes métier sont montées dans `src/app.js` avec un préfixe logique: `/user`, `/events`, `/qr`, `/dashboard`, `/areas`, `/agents` et `/export`. La séparation des responsabilités est simple et doit être conservée: les routes ne font que router, les contrôleurs orchestrent la requête, les services portent le métier et Prisma persiste.

Routes	Fichiers <code>src/routes/*.routes.js</code> . Ils déclarent les middlewares d'authentification et de rôles puis appellent les contrôleurs.
Contrôleurs	Fichiers <code>src/controllers/api/*.controller.js</code> . Ils vérifient <code>req.user</code> , <code>params</code> et <code>body</code> , puis retournent JSON ou fichiers.
Services	Fichiers <code>src/services/*.service.js</code> . Ils encapsulent Prisma et les règles importantes comme la validation des zones ou la décision de scan.
Middleware auth	<code>authMiddleware.js</code> lit le Bearer token ou le cookie token, vérifie <code>PUBLIC_KEY</code> en RS256 et exige <code>token_type=access</code> .
Middleware rôles	<code>roleMiddleware.js</code> limite certaines routes à <code>ORG_ADMIN</code> et <code>SUPER_ADMIN</code> .
Logger	<code>utils/logger.js</code> produit des logs JSON et masque <code>password</code> , <code>token</code> , <code>secret</code> , <code>authorization</code> , <code>cookie</code> et <code>cles</code> . Le backend sert aussi les fichiers statiques de <code>src/statics</code> . Les QR physiques sont créés dans <code>src/statics/qrcodes</code> et les cartes SVG dans <code>src/statics/cards</code> . Le frontend reçoit des URLs relatives comme <code>/qrcodes/qr_TOKEN.png</code> ou <code>/cards/card_TOKEN.svg</code> et doit les prefixer avec <code>NEXT_PUBLIC_API_URL</code> si l'affichage se fait depuis un autre domaine.

Attention

Ne pas dupliquer la logique métier dans le frontend. Par exemple, l'UI peut afficher un statut lisible, mais la décision réelle d'autoriser ou refuser un scan doit rester dans le backend, notamment dans `qr_policy.service.js`.

4. Modèle de données PostgreSQL/Prisma

Le schéma principal est `BackendAccessQ/src/prisma/schema.prisma`. Le provider est `postgresql` et l'URL vient de `DATABASE_URL`. Les noms Prisma sont parfois différents des noms SQL grâce aux `@@map`: `UserQ` correspond à `usersQ`, `QrCode` à `qr_codes`, `Area` à `area`, `Plan` à `plan`.

Relations principales

- Organization 1-N UserQ: une organisation possède ses administrateurs, agents et opérateurs.
- Organization 1-N Event: chaque événement appartient à une organisation.
- Organization 1-N Area: chaque zone appartient à une organisation.
- Event 1-N QrCode: un QR code est généré pour un événement.
- QrCode 1-N ScanLog: chaque scan journalise référence un QR.
- UserQ 1-N ScanLog: l'utilisateur qui scanne est conservé dans `scanned_by_id`.
- Event N-N Area via EventSchedule: un événement peut couvrir plusieurs zones, avec dates de début et fin.

Tables et champs critiques

UserQ	<code>user_id</code> , <code>clef</code> , <code>full_name</code> , <code>email</code> , <code>password_hash</code> , <code>role</code> , <code>is_verified</code> , <code>is_active</code> , <code>verification_token</code> , <code>deleted_at</code> , <code>created_at</code> , <code>last_login</code> , <code>org_id</code> .
--------------	---

Organization	org_id, name, created_at, deleted_at, is_active, subscription_plan.
Event	event_id, org_id, title, description, created_at, deleted_at.
Area	area_id, org_id, area_name, accreditation_level, deleted_at.
EventSchedule	id, id_event, id_area, start_date, end_date. C'est le lien evenement-zone avec periode.
QrCode	qr_id, event_id, unique_token, status, usage_limit, scans_count, valid_from, valid_until, deleted_at, level, holder_name, holder_email, holder_phone.
ScanLog	id, qr_code_id, scanned_by_id, scanned_at, location_lat, location_long, status.
Plan	plan_id, title, cost, features JSON.

Enums

```

Role: SUPER_ADMIN, ORG_ADMIN, ORG_AGENT, OPERATOR
QrStatus: active, revoked, expired, used_up
ScanStatus: authorized, denied_expired, denied_revoked, denied_limit_reached

```

Le projet privilegie le soft-delete. Pour les organisations, evenements, zones, utilisateurs et QR codes, un deleted_at non nul signifie que l'objet est archive ou supprime logiquement. Plusieurs requetes filtrent deleted_at:null pour masquer ces objets sans detruire l'historique. La suppression d'un evenement marque aussi ses QR codes comme revoked et les soft-delete dans une transaction.

Attention

AdministrationAccess possede aussi un schema Prisma. Le schema backend est le plus complet actuellement, notamment avec des index et deleted_at sur Area. Lors d'une migration, verifier ou synchroniser les deux schemas pour eviter une divergence silencieuse.

5. Authentication, roles et sessions

L'authentification de l'API principale repose sur deux JWT: un access token court et un refresh token plus long. Les tokens sont signes avec PRIVATE_KEY en RS256 et verifies avec PUBLIC_KEY. Le payload contient user_id, email, role, org_id et token_type. authMiddleware refuse les refresh tokens sur les routes metier car il exige token_type=access.

Cookies web

Lors d'un login reussi, le backend pose deux cookies HttpOnly: token et refreshToken. Les options de cookies viennent de config/security.js. En production, secure devient vrai par default, et SameSite devient none si necessaire. Le frontend Next envoie credentials: include dans apiFetch. En cas de 401/403 d'authentification, il tente /user/refresh puis rejoue la requete.

Inscription et verification email

POST /user/signin exige fullName, email, organizationName et password. Le mot de passe doit respecter la politique PasswordValidator. Si l'email est libre, le service cree une Organization et un UserQ ORG_ADMIN dans une transaction. Le compte est cree avec is_verified=false et un verification_token. L'email de verification pointe vers FRONTEND_URL/verify-email?token=TOKEN.

Agents et permissions

Un ORG_ADMIN ou SUPER_ADMIN peut inviter un agent via /agents/add-agent. Le mot de passe est fourni ou genere automatiquement, hash avec bcrypt, et l'utilisateur est cree comme ORG_AGENT, OPERATOR ou ORG_ADMIN selon le role demande. Les agents invites sont is_verified=true car l'invitation est interne.

Attention

Le code de login verifie deleted_at et is_verified, mais ne verifie pas partout is_active. Or AdministrationAccess peut desactiver une organisation ou un utilisateur via is_active=false. Si l'on veut que la desactivation bloque toutes les connexions, il faut renforcer la verification cote backend principal.

6. Contrats d'API

Les reponses JSON suivent majoritairement la forme { success: boolean, message?: string, data?: object }. Les erreurs metier utilisent souvent 400, 403 ou 404. Certains refus de scan retournent HTTP 200 avec success:false pour simplifier l'UX du scanner.

Routes utilisateur

POST	/user/login	Body email,password. Pose token et refreshToken. Retourne success, token et user.
POST	/user/refresh	Utilise le cookie refreshToken. Repose une session et retourne un access token.
POST	/user/signin	Body fullName,email,organizationName,password. Cree organisation + ORG_ADMIN non verifie.
POST	/user/verify-email	Body token. Active is_verified et nettoie verification_token.
GET	/user/profile	Retourne user_id,email,full_name,role,org_id du compte connecte.
PUT	/user/profile	Body fullName,email. Verifie l'unicite email.
PUT	/user/password	Change le mot de passe du compte connecte.
GET	/user/org	Admin only. Lit l'organisation courante.
PUT	/user/org	Admin only. Modifie l'organisation courante.
DELETE	/user/org	Admin only. Soft-delete organisation et utilisateurs.
GET	/user/logout	Efface token et refreshToken.

Routes evenements

GET	/events?search=q	Liste les evenements de l'organisation avec zones et nombre de QR actifs.
GET	/events/:event_id	Detail d'un evenement de l'organisation.
POST	/events	Admin only. Body title,description,arealds ou id_area,startDate,endDate.
PUT	/events/:event_id	Admin only. Met a jour texte, dates et zones.
DELETE	/events/:event_id	Admin only. Soft-delete et revocation des QR de l'evenement.

Routes zones et agents

GET	/areas	Liste les zones non supprimees de l'organisation.
GET	/areas/:id	Detail d'une zone de l'organisation.
POST	/areas	Admin only. Body area_name, accreditation_level.
PUT	/areas/:id	Admin only. Met a jour une zone existante.
DELETE	/areas/:id	Admin only. Soft-delete une zone.
GET	/agents	Liste ORG_AGENT, ORG_ADMIN et OPERATOR de l'organisation avec nombre de scans.

POST	/agents/add-agent	Admin only. Body fullName,email,role,password optionnel.
PUT	/agents/:id/toggle	Admin only. Active/desactive par deleted_at; refuse son propre compte et les ORG_ADMIN.
DELETE	/agents/:id	Admin only. Soft-delete un agent non admin.
Routes QR, dashboard et exports		
GET	/qr/qrs	Tous les QR de l'organisation, formates pour l'UI.
GET	/qr/event/:event_id	QR d'un evenement de l'organisation.
POST	/qr/generate/:event_id	Body fullName,email,phone,accessType,limit,validFrom,validUntil,level,cardTemplateId.
GET	/qr/template/:event_id	Telecharge un modele CSV d'import.
POST	/qr/import/:event_id	Upload multipart file CSV, taille max 5 MB.
PUT	/qr/revoke/:id	Passe status a revoked apres verification d'organisation.
POST	/qr/verify	Body token, location optionnel. Cree ScanLog selon la decision.
GET	/dashboard/stats	Stats activeQrs,totalScans,upcomingEvents,nextEventTitle,activeAgents,recentScans,scansByDay,topAgents.
GET	/export/csv?event_id=	Export CSV des scans de l'organisation, optionnellement pour un evenement.
GET	/export/pdf?event_id=	Export PDF des 100 derniers scans, optionnellement pour un evenement.

7. Flux metier details

Creation d'organisation

Le visiteur s'inscrit depuis frontendAccessQ/src/app/register/page.jsx. Le frontend appelle POST /user/signin. Le backend verifie les champs, valide le mot de passe, hash avec bcrypt, cree une organisation et son administrateur dans une transaction, genere verification_token et envoie un email Brevo. Tant que l'email n'est pas verifie, /user/login retourne un refus.

Creation d'evenement

Le frontend charge d'abord /areas afin d'afficher les zones disponibles. Lors de la soumission, il exige title, startDate, endDate et au moins une zone. Le backend verifie a nouveau que les zones appartiennent a l'organisation via assertAreasBelongToOrg. La creation est transactionnelle: l'evenement et ses EventSchedules sont crees ensemble.

Mise a jour d'evenement

Le controller verifie d'abord que l'evenement appartient a org_id. Le service updateEvent peut mettre a jour title/description, synchroniser les zones si arealds ou id_area sont fournis, ou seulement modifier les dates des schedules existants. Si les zones changent, les anciens EventSchedules sont supprimes puis recrees.

Generation d'un QR individuel

POST /qr/generate/:event_id verifie que l'evenement appartient a l'organisation. Le backend cree uniqueToken avec crypto.randomUUID, determine usage_limit selon accessType, sauvegarde QrCode, genere un PNG QR avec correction d'erreur H, puis retourne qrUrl et event. Si cardTemplateId est fourni et valide, il genere aussi une carte SVG.

Contenu encode dans l'image QR:

```
{
  "t": "unique_token_du_qr",
  "e": 123
}
```

Le scanner extrait t. Les donnees du detenteur restent en base.

Import CSV

POST /qr/import/:event_id accepte un fichier CSV seulement, limite a 5 MB. Les colonnes reconnues sont fullName, name, nom, email, phone, telephone, accessType, limit, validFrom, validUntil et level. Chaque ligne avec un nom genere un QrCode et un PNG. Le fichier temporaire upload est supprime apres le traitement.

Verification de scan

Le frontend scan/page.jsx charge la librairie html5-qrcode depuis <https://unpkg.com/html5-qrcode>. Apres lecture, il arrete la camera, tente de parser le QR comme JSON, extrait parsed.t si present, recupere la geolocalisation si le navigateur l'autorise, puis appelle POST /qr/verify.

Le controller charge le QR par unique_token avec son evenement et son organisation, puis delegue la decision a evaluateQrScan. Si shouldRecord=false, aucun ScanLog n'est cree. Si shouldRecord=true, recordScan cree un ScanLog et incremente scans_count uniquement pour authorized. Si la limite vient d'etre atteinte, le QR passe a used_up.

Ordre de decision evaluateQrScan:

1. QR inexistant -> 404, pas de journal.
2. QR/evenement/organisation supprime ou organisation inactive -> 410, pas de journal.
3. QR d'une autre organisation -> 403, pas de journal.
4. status revoked -> denied_revoked, journal.
5. scans_count >= usage_limit -> denied_limit_reached, journal.
6. maintenant < valid_from -> denied_expired, journal.
7. maintenant > valid_until -> denied_expired, journal.
8. sinon authorized, journal, increment scans_count.

Attention

La fonction evaluateQrScan est l'endroit le plus important pour les regles d'accès. Toute modification doit être accompagnée de tests car elle impacte directement la sécurité terrain.

8. Frontend Next.js

Le frontend repose sur Next.js 16, React 19, Tailwind CSS 4, lucide-react et Recharts. L'application utilise des composants client avec useState/useEffect pour appeler l'API. Les pages principales sont organisées dans frontendAccessQ/src/app.

app/page.js	Page d'accueil.
login/page.jsx	Connexion. Appelle directement /user/login avec credentials:include et redirige vers /dashboard.
register/page.jsx	Creation d'organisation et administrateur.
verify-email/page.jsx	Lit le token depuis l'URL et appelle /user/verify-email.
dashboard/layout.jsx	Shell principal: sidebar, navigation, profil, deconnexion. Charge /user/profile.
dashboard/page.jsx	Tableau de bord: charge /user/profile et /dashboard/stats, exporte CSV/PDF.
dashboard/events/page.jsx	Liste, recherche et filtre les evenements. Charge /events et /user/profile pour le role.
dashboard/events/new/page.jsx	Creation d'evenement. Charge /areas puis POST /events.
dashboard/events/[id]/page.jsx	Detail evenement, gestion des QR et actions associees.
dashboard/areas/page.jsx	CRUD zones via /areas.
dashboard/agents/page.jsx	Gestion agents via /agents.
dashboard/card-templates/page.jsx	Choix des modeles de cartes cote UI.
dashboard/settings/page.jsx	Profil, mot de passe et organisation.
scan/page.jsx	Scanner terrain avec camera, geolocalisation optionnelle et /qr/verify.

Wrapper API

frontendAccessQ/src/app/lib/api.js centralise les appels authentifis. apiFetch prefixe les chemins avec NEXT_PUBLIC_API_URL, ajoute credentials:include, tente un refresh en cas de 401/403 d'authentification et redirige vers /login?next=... si la session est invalide. Utiliser ce wrapper pour toute nouvelle page authentifiee.

Gestion des roles cote UI

Le layout masque Agents pour les roles qui ne sont ni SUPER_ADMIN ni ORG_ADMIN. Les pages evenements montrent ou masquent les actions d'administration selon le role. Ce masquage ameliore l'UX mais ne doit jamais remplacer les controles backend.

Attention

La page events/page.jsx filtre avec les valeurs Upcoming, Active et Past, alors que le backend renvoie actuellement des libelles francais comme Actif, A venir et Passe. Cela peut rendre le filtre de statut incoherent. A corriger en uniformisant les valeurs machine et les libelles affichees.

9. Administration super-admin

AdministrationAccess est une application Express/EJS sur PORT 4000 par default. Elle sert des vues depuis src/views et des assets depuis src/public. Elle utilise cookie-parser, json/urlencoded et Prisma. La connexion se fait via /login avec un compte UserQ dont role vaut SUPER_ADMIN.

Contrairement au backend principal, AdministrationAccess signe son adminToken avec JWT_SECRET et non avec PRIVATE_KEY/PUBLIC_KEY. Le token dure 1 jour et est stocke en cookie HttpOnly. requireAuth verifie le token et redirige vers /login si le role n'est pas SUPER_ADMIN.

- GET /dashboard: affiche totalUsers, totalOrganizations et totalEvents.
- GET /organizations: liste les organisations avec nombre d'utilisateurs et evenements.

- POST /organizations/:id/deactivate: is_active=false sur l'organisation et ses utilisateurs.
- POST /organizations/:id/activate: is_active=true sur l'organisation et ses utilisateurs.
- POST /organizations/:id/archive: deleted_at sur l'organisation et ses utilisateurs.
- GET /users: liste les utilisateurs avec nom d'organisation.
- POST /users/:id/deactivate: is_active=false sur l'utilisateur.
- POST /users/:id/activate: is_active=true sur l'utilisateur.

Note de reprise

Le backend de scan tient compte de organization.is_active=false dans evaluateQrScan. En revanche, la connexion principale devrait aussi controler is_active pour que la desactivation soit complete.

10. Emails, QR physiques et cartes

Email transactionnel

BackendAccessQ/src/services/email.service.js utilise l'API Brevo v3 via axios. Les variables attendues sont BREVO_API_KEY, BREVO_SENDER_NAME et BREVO_SENDER_EMAIL. Le service envoie deux types d'emails: verification d'adresse pour l'inscription et invitation agent avec mot de passe temporaire. Les erreurs d'envoi sont capturees et loguees; les fonctions retournent false mais les controllers ne bloquent pas toujours le flux sur cet echec.

QR physiques

Les QR sont generes avec le package qrcode. Le fichier est un PNG de 400px, marge 2, correction d'erreur H. Le chemin disque est src/statics/qrcodes/qr_TOKEN.png et l'URL publique est /qrcodes/qr_TOKEN.png.

Cartes SVG

card_template.service.js contient trois modeles: event-ticket, staff-card et wedding-invite. Chaque modele est rendu en SVG, avec les informations evenement, detenteur, zone, date, niveau et image QR. Le fichier produit est src/statics/cards/card_TOKEN.svg et l'URL est /cards/card_TOKEN.svg.

Attention

Les cartes SVG utilisent une balise image href vers qrUrl, souvent une URL relative /qrcodes/.... Si les cartes sont ouvertes hors du contexte du backend, il peut etre necessaire d'utiliser une URL absolue.

11. Configuration, installation et deploiement

Variables backend

```
NODE_ENV=development|production
PORT=5000
DATABASE_URL=postgresql://USER:PASSWORD@HOST:PORT/DB
FRONTEND_URL=http://localhost:3000
ADMIN_URL=http://localhost:4000
CORS_ORIGINS=https://autre-domaine.example
TRUST_PROXY=false|true|nombre
COOKIE_SAMESITE=lax|strict|none
COOKIE_SECURE=true|false
PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n..."
PUBLIC_KEY="-----BEGIN PUBLIC KEY-----\n..."
JWT_SECRET=secret-long-pour-admin-ou-compat
ACCESS_TOKEN_EXPIRES_IN=15m
REFRESH_TOKEN_EXPIRES_IN=7d
SALT_ROUNDS=10
BREVO_API_KEY=...
BREVO_SENDER_NAME=QR Access
BREVO_SENDER_EMAIL=no-reply@example.com
LOG_LEVEL=debug|info|warn|error|silent
LOG_REQUESTS=true|false
SLOW_REQUEST_MS=1000
```

Commandes locales

```
# Backend
cd BackendAccessQ
npm install
cp .env.example .env
npx prisma generate
npx prisma migrate deploy
npm start

# Frontend
cd frontendAccessQ
npm install
echo NEXT_PUBLIC_API_URL=http://localhost:5000 > .env.local
npm run dev

# Administration
cd AdministrationAccess
npm install
cp .env.example .env
npm start
```

Production

En production, le backend doit être servi en HTTPS ou derrière un proxy qui termine TLS. Si frontend et backend sont sur des domaines différents, configurer CORS avec le domaine exact du frontend, COOKIE_SAMESITE=none et COOKIE_SECURE=true. TRUST_PROXY doit correspondre à l'infrastructure afin que les cookies secure, rate limiters et logs IP fonctionnent correctement.

PostgreSQL est obligatoire. Les migrations du backend sont dans BackendAccessQ/src/prisma/migrations. Utiliser prisma migrate deploy pour appliquer les migrations déjà versionnées en production. Éviter db push en production sauf décision explicite car il contourne l'historique de migrations.

Attention

Les fichiers QR et cartes sont generes sur le disque local de l'application. Sur une plateforme avec filesystem ephemere ou instances multiples, il faudra migrer ces assets vers un stockage persistant partage, par exemple S3, Cloudinary ou volume persistant.

12. Tests, logs et diagnostic

BackendAccessQ utilise le runner natif Node avec npm test, qui execute node --test. Les fichiers test couvrent l'auth middleware, les routes QR, zones, evenements, agents, utilisateurs, exports, la configuration securite, le logger, la policy QR et certains services Prisma.

```
cd BackendAccessQ
npm test

cd frontendAccessQ
npm run lint
npm run build
```

Logs

Les logs sont en JSON et contiennent ts, level, event, service et les metas. Le logger masque automatiquement les champs sensibles. Les events importants incluent cors.denied, auth.denied, session.refreshed, session.logout, event.created, event.updated, event.deleted, area.created, agent.created, qr.scan_authorized, qr.scan_denied et qr.scan_rejected.

Procedure de diagnostic rapide

- Erreur CORS: verifier FRONTEND_URL, ADMIN_URL, CORS_ORIGINS et l'origine exacte dans le navigateur.
- Session qui expire trop vite: verifier ACCESS_TOKEN_EXPIRES_IN, refreshToken, cookies HttpOnly et SameSite/Secure.
- Scan refuse a tort: verifier unique_token, org_id de l'agent, status QR, usage_limit, scans_count, valid_from, valid_until et deleted_at.
- QR image introuvable: verifier src/statics/qrcodes, l'URL publique et le fait que le backend serve express.static(src/statics).
- Agent ne peut pas se connecter: verifier deleted_at, is_verified, role, hash mot de passe et eventuellement is_active.
- Stats dashboard incoherentes: verifier les requetes Prisma dans api.dashboard.controller.js et les filtres org_id.
- Exports vides: verifier l'existence de ScanLog et le filtre event_id.

Tests a ajouter en priorite

- Tests de login avec is_active=false pour verrouiller le comportement attendu.
- Tests frontend ou integration pour le refresh automatique apiFetch.
- Tests d'import CSV avec lignes invalides, dates invalides et accessType unlimited.
- Tests de generation de carte SVG avec URL absolue si le stockage change.
- Tests d'autorisation par role sur chaque route admin.

13. Maintenance et evolutions recommandees

Regles a ne pas casser

- Chaque requete metier doit rester filtree par org_id.
- Le token QR reste un identifiant opaque; ne pas encoder les donnees personnelles dans le QR.
- Les suppressions doivent rester logiques sauf besoin legal ou operation de purge controlee.
- La decision de scan doit rester centralisee et testee.
- Les roles UI ne remplacent jamais les roles backend.
- Les migrations Prisma doivent etre versionnees et appliquees dans l'ordre.
- Les assets generes doivent avoir une strategie de persistance en production.

Dette technique visible

- Deux schemas Prisma coexistent entre BackendAccessQ et AdministrationAccess.
- is_active n'est pas applique de facon uniforme dans tous les flux d'authentification.
- Les valeurs de statut evenement cote frontend ne semblent pas totalement alignees avec le backend.
- Le scanner depend d'un CDN externe charge a l'execution; prevoir une dependance package si l'usage offline/

PWA devient important.

- Les QR et cartes sont stockes localement; c'est fragile en production scalable.
- Les emails d'invitation contiennent un mot de passe temporaire en clair; idealement basculer vers un lien de definition de mot de passe.
- L'export PDF est volontairement simplifie et limite a 100 scans; il faudra l'etendre pour des rapports complets.

Roadmap technique conseillee

A court terme, uniformiser les statuts et renforcer is_active dans le backend principal. Ensuite, centraliser le schema Prisma ou automatiser sa synchronisation avec AdministrationAccess. Pour la production, externaliser les assets generes, stabiliser les variables cookies/CORS et ajouter des tests d'integration couvrant les principaux parcours: inscription, creation evenement, generation QR, scan autorise, scan refuse, export.

A moyen terme, separer les permissions de facon plus declarative, ajouter une couche d'audit pour les actions sensibles, rendre les exports pages ou asynchrones, et remplacer les mots de passe temporaires en email par un flux d'invitation securise. Si l'application doit fonctionner sur site avec peu de reseau, integrer html5-qrcode au bundle frontend plutot que de le charger depuis un CDN.

14. Resume pour un nouveau preneur

Pour comprendre AccessQ, commencer par le schema Prisma, puis lire app.js pour voir les middlewares globaux, ensuite les routes et controllers QR/evenements/utilisateurs. Le flux le plus critique est: login -> creation evenement et zones -> generation QR -> scan -> ScanLog -> dashboard/export. Si ce flux est compris, le reste de l'application devient beaucoup plus lisible.

Le backend est le gardien de la securite et de la decision. Le frontend est une interface riche qui consomme les routes et gere le confort utilisateur. AdministrationAccess gere la plateforme au-dessus des organisations. Toute evolution doit respecter cette repartition pour que le projet reste maintenable.